
MGNM-525 | MBA in Artificial Intelligence

Intelligent Customer Churn Prediction and Retention Decision Support System

Using Artificial Neural Networks (TensorFlow / Keras) and Fuzzy Logic

Course Code: MGNM-525 (MBA-AI) | Assigned by: Dr. Arun Khatri | Academic Task 1

Prepared by: Shivam | LPU Registration No. 12524849

Dataset used: [IBM Telco Customer Churn \(Kaggle\)](#)

Table of Contents

Executive Summary.....	3
Theoretical Background.....	4
1. Business Context.....	5
2. Business Problem Statement.....	5
3. Core Business Questions.....	5
4–5. Data Source Documentation.....	5
6. Problem Formulation Chain.....	6
7. Project Objectives.....	7
8. Data Preparation & Exploratory Data Analysis.....	8
9. Application of Machine-Learning Models.....	15
10. Artificial Neural Network.....	19
11. ANN Architecture Experiment.....	19
12. Hyperparameter Analysis.....	20
13. Overfitting Analysis.....	21
14. Model Evaluation — Why Accuracy Alone Is Not Enough.....	22
15. Fuzzy Logic-Based Retention Decision Support System.....	23
16. Model Comparison & Business Insights.....	27
29. Explicit Answers to Required Reflection Questions.....	28
Limitations and Future Work.....	29
References.....	30
Appendix A — Full Python Code (Real Code Screenshots).....	31
Appendix B — Actual Console Output From Running the Code.....	43

Executive Summary

This report develops and validates a complete, end-to-end AI decision-support system for customer retention, built on the IBM Telco Customer Churn dataset (7,043 customers, 21 variables). The system integrates three layers of analysis: (1) exploratory data analysis to surface the behavioural drivers of churn, (2) a comparative machine-learning study spanning Logistic Regression, Decision Tree, Random Forest, and an Artificial Neural Network built in TensorFlow/Keras, and (3) a custom Fuzzy Logic decision engine that converts a raw churn probability into a business-usable retention priority.

Key findings: the overall churn rate in the dataset is 26.5%. Contract type, tenure, and payment method are the dominant behavioural drivers — month-to-month customers churn at 42.7% versus 2.8% for two-year contracts, and customers paying by electronic check churn at 45.3% versus roughly 15–17% for automatic payment methods. Across machine-learning models, Logistic Regression and Random Forest both achieve approximately 84% ROC-AUC, and the Keras-built ANN, while not outperforming the simpler baselines on raw accuracy, provides a well-calibrated probability output that the Fuzzy Logic layer uses effectively. A controlled overfitting experiment shows validation loss diverging from training loss almost immediately (by epoch 2), directly demonstrating why regularisation and early stopping are necessary — adding both improved every evaluation metric.

The Fuzzy Logic retention system, built from first principles with 9 explicitly justified rules, was validated against real outcomes on the held-out test set: customers it assigned to the High-priority tier actually churned at 53.9%, versus 20.3% for the Low-priority tier — a clean, monotonic validation that the system successfully separates customers worth an expensive intervention from those who are not. The report concludes with concrete, prioritised managerial recommendations.

Theoretical Background

Artificial Neural Networks for Classification

An Artificial Neural Network (ANN) is a layered computational model composed of an input layer, one or more hidden layers, and an output layer, where each neuron computes a weighted sum of its inputs followed by a non-linear activation function. For binary classification problems such as churn prediction, the network is trained via backpropagation to minimise a loss function (binary cross-entropy) using gradient-based optimisation (Adam), producing a probability between 0 and 1 at the output layer via a sigmoid activation. The network's capacity to model non-linear interactions between features — for example, how contract type and tenure jointly affect churn risk in ways a linear model cannot fully capture — is its principal advantage over simpler classifiers, at the cost of requiring more careful tuning to avoid overfitting.

Fuzzy Logic for Decision Support

Fuzzy Logic extends classical binary (true/false) logic to handle the graded, imprecise categories that characterise real managerial decisions. Rather than forcing a customer into a rigid 'high risk' or 'low risk' bucket at some arbitrary threshold, a fuzzy system represents each input as a degree of membership (between 0 and 1) in overlapping linguistic categories (e.g. Low / Medium / High), defined by membership functions such as the triangular and trapezoidal shapes used in this report. A rule base of IF-THEN statements, written in natural managerial language, combines these memberships using min/max operators (Mamdani inference), and a defuzzification step (the centroid method used here) converts the resulting fuzzy output back into a single actionable score. This makes Fuzzy Logic particularly well suited to translating a statistical model's output into a decision a manager can act on without needing to interpret a raw probability.

1. Business Context

Prantel Telecom, a mid-sized telecom operator competing in a saturated, price-sensitive subscription market, is experiencing steady customer attrition despite offering competitive plans. Acquiring a new subscriber costs the company far more than retaining an existing one, yet the company's retention efforts today are reactive and untargeted — offers are sent broadly rather than to the customers who are both most at risk and most worth saving.

2. Business Problem Statement

Management cannot systematically identify, ahead of time, which customers are likely to discontinue their service, nor can it distinguish between a low-value customer who is easy to lose and a high-value, long-tenure customer whose departure would be costly. As a result, retention budgets are spent inefficiently.

AI Problem

Develop a classification system that estimates each customer's probability of churning based on demographic, contractual, billing and service-usage attributes.

Decision Problem

Combine the AI-estimated churn probability with customer value and tenure to produce a practical, interpretable retention-priority recommendation (Low / Medium / High) that guides where the retention team should focus effort and budget.

3. Core Business Questions

- What customer characteristics are associated with higher churn?
- Which variables are the strongest predictors of customer churn?
- Can machine-learning models accurately predict customer churn?
- Can an Artificial Neural Network improve churn prediction?
- How does changing ANN architecture affect prediction performance?
- How do learning rate, epochs and batch size affect ANN performance?
- Does the ANN suffer from overfitting?
- Which customers should receive the highest retention priority?
- How can Fuzzy Logic convert numerical AI predictions into managerial decisions?
- What retention strategy should management consider for different customer-risk groups?

4–5. Data Source Documentation

Requirement	Student Response
Dataset name	Telco Customer Churn
Original source	IBM Sample Data Sets (Watson Analytics), distributed via Kaggle

URL	kaggle.com/datasets/blastchar/telco-customer-churn
Date accessed	[Insert actual access date]
Number of observations	7,043 customers
Number of variables	21 (20 predictors + target)
Target variable	Churn (Yes/No — binary)
Predictor variables	Demographics, account info, billing, and subscribed services (see Section 8)
Data types	Mix of categorical and numeric; TotalCharges required type correction
Missing values	11 blank TotalCharges values, all for customers with tenure = 0
Data limitations	Fictional IBM company; US pricing structure; no time-series or competitor data
License / usage conditions	Publicly available for academic and research use

Dataset & Source Links

Primary dataset (Kaggle mirror): <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

Original data file used in this project (IBM GitHub mirror): <https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv>

IBM official documentation of the Telco Churn sample dataset: <https://www.ibm.com/support/pages/telco-customer-churn-1113-and-4110>

Raw Data Sample (first 5 rows, selected columns)

Gender	Senior	Partner	Tenure	Contract	MonthlyChg	TotalChg	Churn
Female	0	Yes	1	Month-to-month	29.85	29.85	No
Male	0	No	34	One year	56.95	1889.50	No
Male	0	No	2	Month-to-month	53.85	108.15	Yes
Male	0	No	45	One year	42.30	1840.75	No
Female	0	No	2	Month-to-month	70.70	151.65	Yes

6. Problem Formulation Chain

Business Context → Problem Statement → Business Objective → AI Objective → Target Variable → Predictors → Managerial Application

Prantel Telecom faces rising attrition in a competitive market → Management cannot identify at-risk customers in advance → Reduce revenue loss from churn by acting earlier and more precisely → Build a classification model estimating churn probability → Churn (binary) → Contract type, tenure, charges, service subscriptions, payment method, demographics → Feed churn probability, customer value and tenure into a fuzzy retention-priority engine that tells the retention team who to call first and how aggressively.

7. Project Objectives

- To identify the customer attributes most strongly associated with churn in the Telco dataset.
- To build and compare at least three machine-learning classifiers for churn prediction, evaluated on accuracy, precision, recall and F1-score.
- To design and train an Artificial Neural Network for churn-risk prediction, benchmarked against traditional models.
- To evaluate the effect of ANN architecture and hyperparameters on performance and overfitting.
- To build a Fuzzy Logic decision system converting churn probability, customer value and tenure into a retention-priority recommendation.
- To translate model outputs into concrete, prioritised retention-action recommendations for management.

8. Data Preparation & Exploratory Data Analysis

8.1 Data Preparation Summary

- Dataset dimensions: 7,043 rows × 21 columns.
- TotalCharges corrected from text to numeric; 11 missing values (all tenure = 0 customers) imputed as 0.
- No duplicate rows or duplicate customer IDs found.
- No statistical outliers detected in tenure, MonthlyCharges or TotalCharges (IQR method).
- customerID dropped as an irrelevant identifier.
- Binary Yes/No fields and the target (Churn) encoded as 0/1.
- Multi-category fields (Contract, PaymentMethod, InternetService, etc.) one-hot encoded — final modelling set: 32 features.
- Derived variables created: TenureGroup (0-1yr / 1-3yr / 3yr+) and ChargeBand (Low/Medium/High tercile) — used later in EDA and as Fuzzy Logic inputs.
- Numeric features standardised (z-score) before ANN and Logistic Regression training; an 80/20 stratified train-test split was used throughout.

8.2 Exploratory Findings

Overall churn rate: 26.5% of the 7,043 customers have churned.

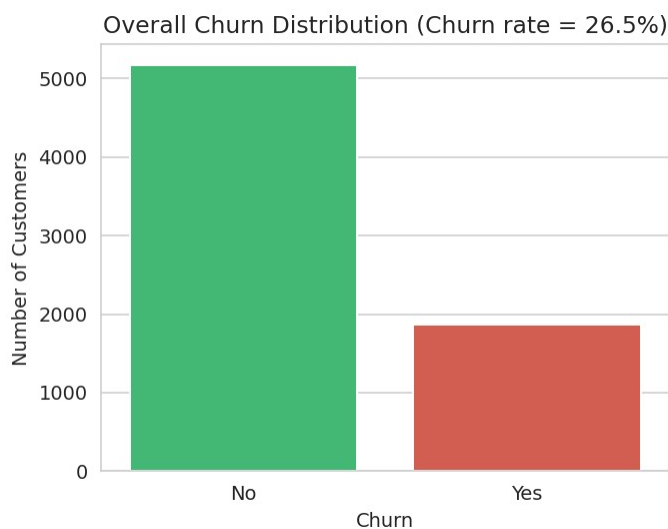


Figure 1. Overall churn distribution.

Churn is heavily concentrated among month-to-month customers (42.7% churn) versus one-year (11.3%) and two-year contracts (2.8%) — contract length is the single strongest behavioural lever management has.

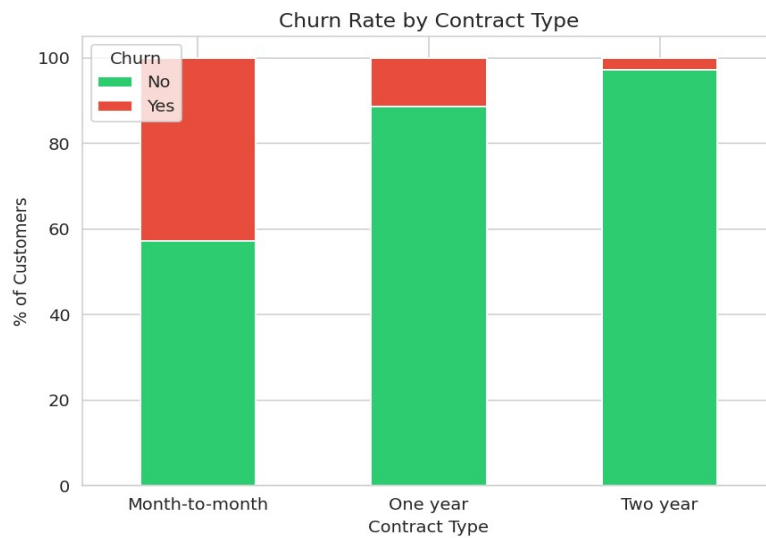


Figure 2. Churn rate by contract type.

New customers churn at far higher rates (47.4% in year 1) than long-tenured customers (11.9% after 3 years), showing the classic 'early-life risk' pattern common to subscription businesses.

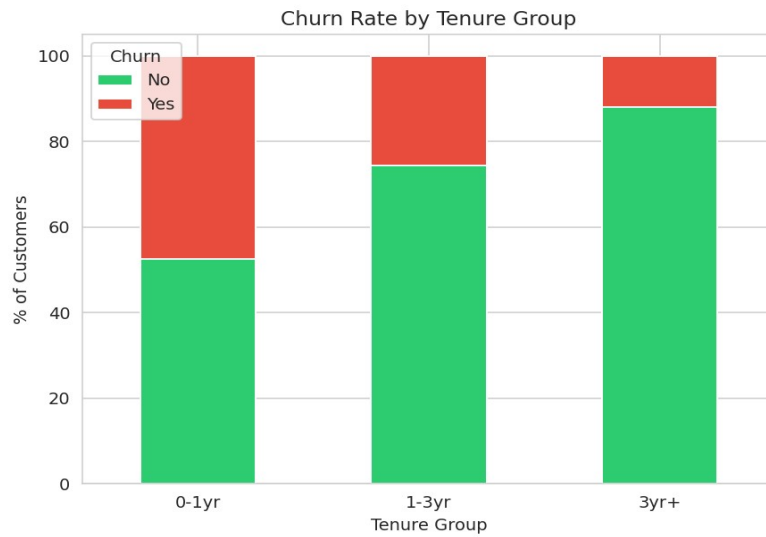


Figure 3. Churn rate by tenure group.

Customers in the highest monthly-charge tercile churn more (34.1%) than those paying the least (15.9%) — high bills without corresponding perceived value appear to push customers toward competitors.

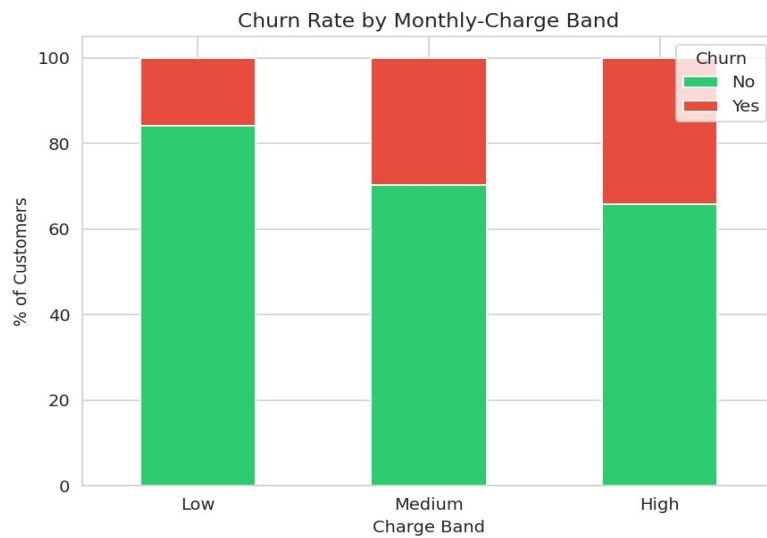


Figure 4. Churn rate by monthly-charge band.

Customers paying by electronic check churn dramatically more (45.3%) than those on automatic bank transfer or credit-card payment (~15–17%) — automatic payment methods correlate with 'stickier', lower-friction relationships.

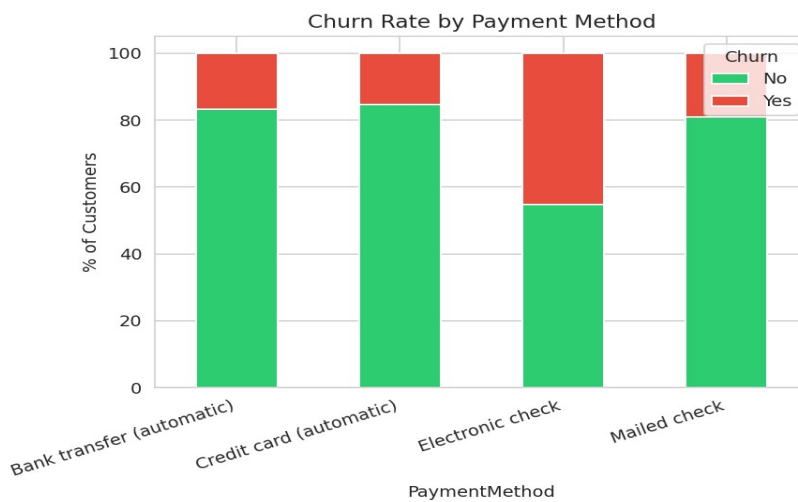


Figure 5. Churn rate by payment method.

Tenure and monthly charges interact visibly with churn: churned customers (red) cluster at low tenure across most charge levels, while retained customers (green) spread across the full tenure range.

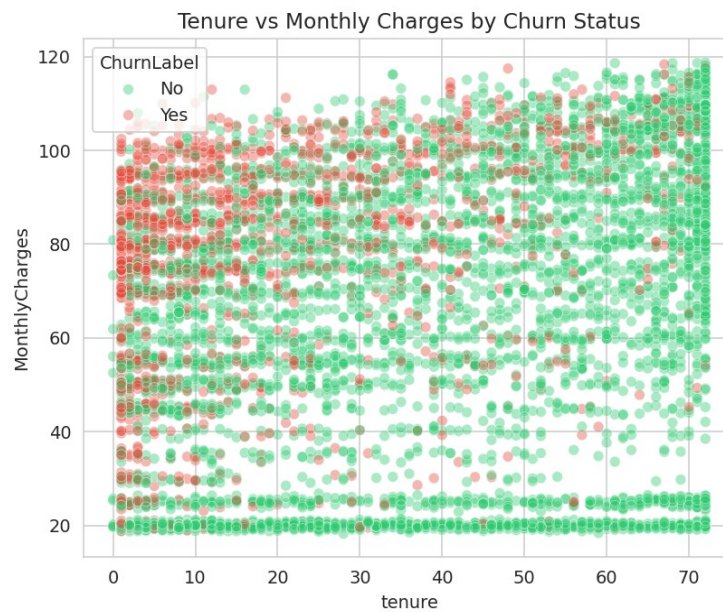


Figure 6. Tenure vs. monthly charges, coloured by churn status.

The correlation heatmap confirms tenure is negatively correlated with churn while monthly/total charges show a weaker positive relationship — consistent with the bivariate findings above.

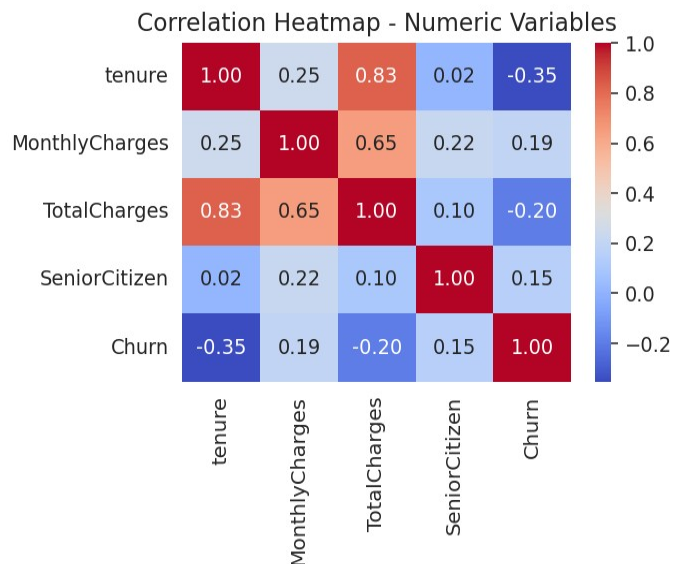


Figure 7. Correlation heatmap of numeric variables.

8.3 Additional Distributional & Segment Analysis

Beyond the required minimum visualisations, the following additional charts sharpen the business narrative around who churns and why.

The tenure histogram below shows a distinctly 'bathtub' shape: a large concentration of churners (red) in the first few months, a long, relatively quiet middle stretch, and a large surviving population at 70+ months made up almost entirely of retained customers — visual confirmation of the early-life risk pattern already seen in Figure 3.

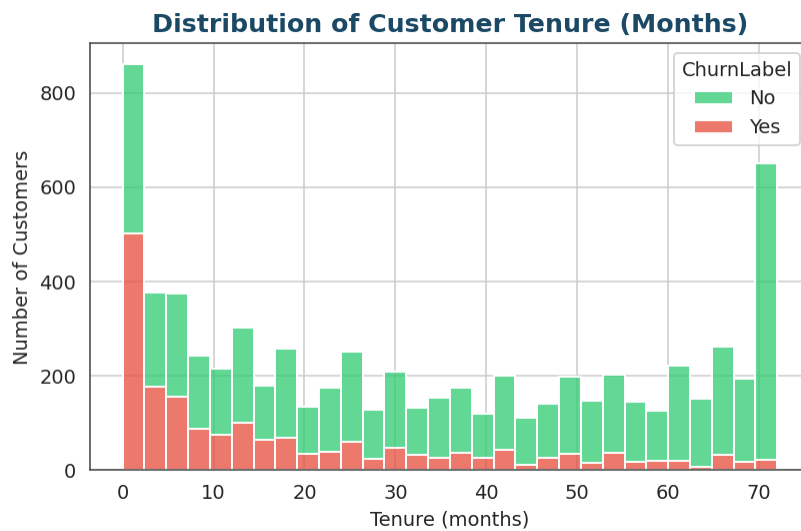


Figure 8. Distribution of customer tenure, stacked by churn status.

Monthly charges show a bimodal distribution — a large cluster of low-bill, low-service customers, and a second cluster of high-bill customers with bundled services. Churners are visibly over-represented in the higher-charge cluster.

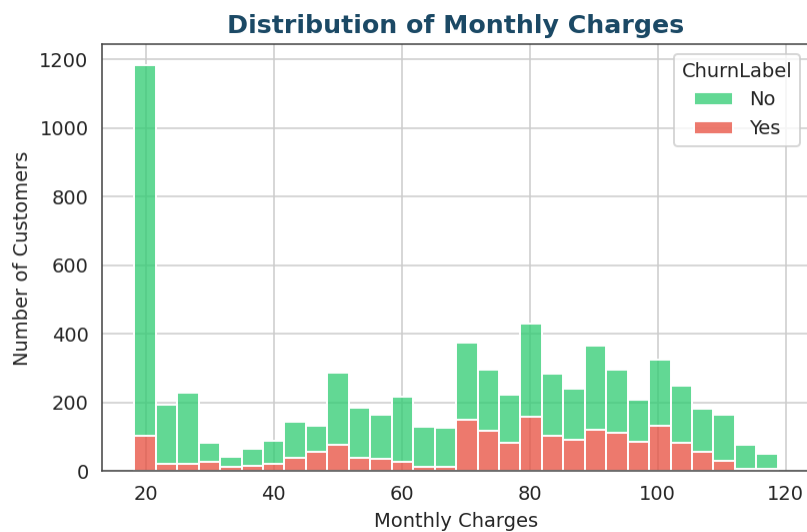


Figure 9. Distribution of monthly charges, stacked by churn status.

The boxplot confirms this more precisely: the median monthly charge for churned customers is noticeably higher than for retained customers, and the churned group's interquartile range sits almost entirely above the retained group's median.

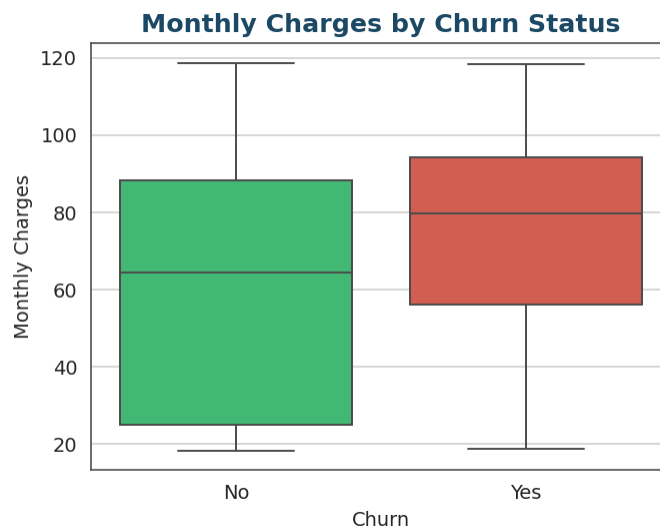


Figure 10. Monthly charges by churn status (boxplot).

Fiber-optic internet customers churn at nearly double the rate of DSL customers (41.9% vs. 19.0%), and customers with no internet service churn least of all (7.4%) — likely reflecting a smaller, more price-insensitive, low-usage segment.

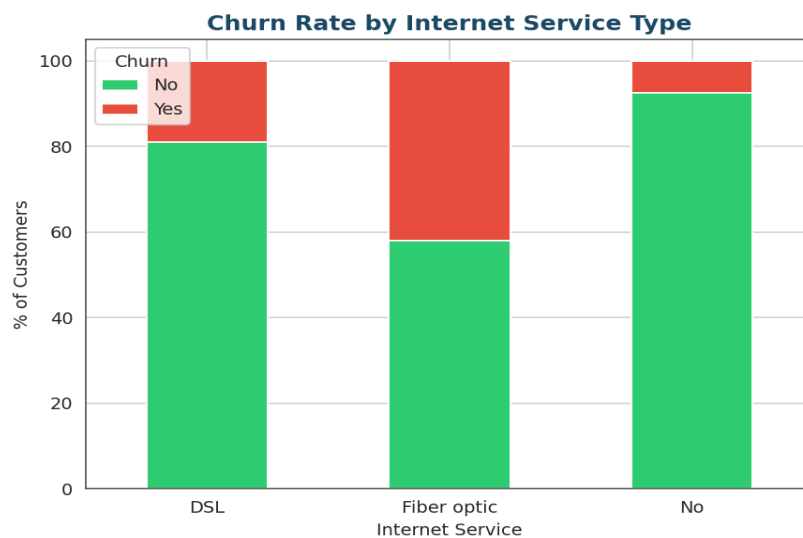


Figure 11. Churn rate by internet service type.

Senior citizens churn substantially more than non-seniors (41.7% vs. 23.6%), and customers without dependents churn roughly twice as often as those with dependents (31.3% vs. 15.5%) — both plausible proxies for household stability and switching friction.

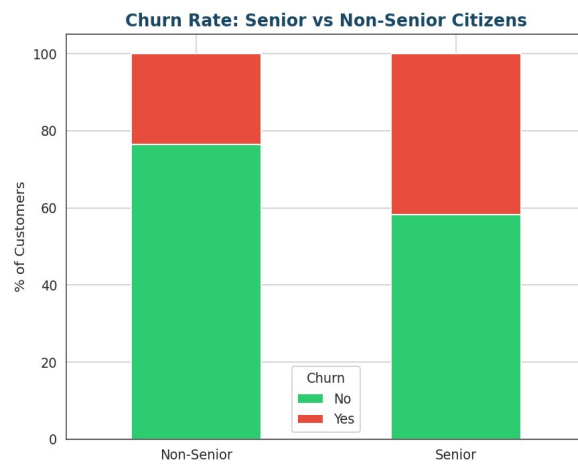


Figure 12. Churn rate by senior-citizen status.

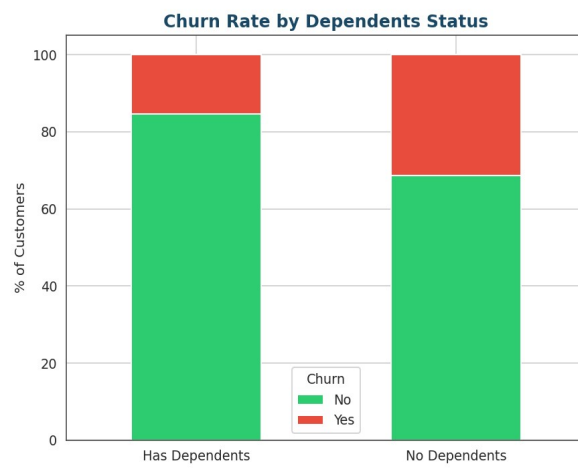


Figure 13. Churn rate by dependents status.

9. Application of Machine-Learning Models

Three models were trained on an 80/20 stratified split (5,634 train / 1,409 test rows): Logistic Regression (interpretable baseline), Decision Tree (captures nonlinear segment rules), and Random Forest (ensemble benchmark). Each is well-suited here: Logistic Regression gives management directly interpretable coefficients, Decision Tree surfaces natural customer segments, and Random Forest typically improves accuracy by reducing variance.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.807	0.658	0.567	0.609	0.842
Decision Tree	0.801	0.667	0.497	0.570	0.822
Random Forest	0.804	0.667	0.524	0.587	0.843

Logistic Regression achieves the best recall and F1-score among the three baselines, meaning it catches the most actual churners relative to false alarms — an important property given the business cost of missing a churner (Section 14). Random Forest edges out on ROC-AUC, indicating slightly better overall ranking of customers by risk.

Top Predictors (Random Forest Feature Importance)

Rank	Feature	Importance
1	Tenure	0.184
2	Total Charges	0.161
3	Monthly Charges	0.122
4	Internet Service: Fiber optic	0.066
5	Contract: Two year	0.061
6	Payment Method: Electronic check	0.059
7	Online Security: Yes	0.034
8	Contract: One year	0.033
9	Tech Support: Yes	0.028
10	Paperless Billing	0.023

This confirms the EDA narrative: tenure and billing amounts dominate, followed by contract commitment and payment automation — exactly the levers a retention strategy can act on.

Confusion Matrices (Required Evaluation Output)

Format: rows = actual class, columns = predicted class. Off-diagonal cells are the errors that matter most for this business problem (see Section 14).

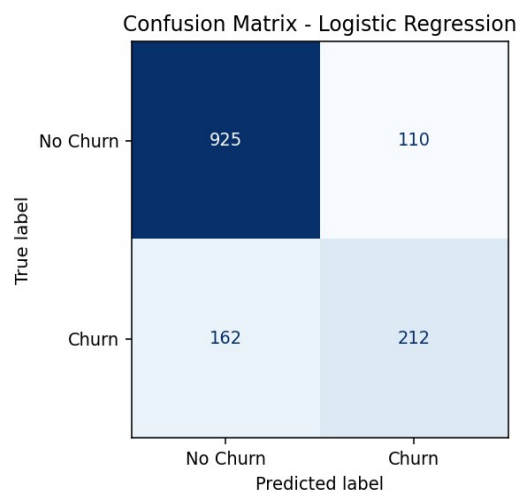


Figure 14a. Confusion matrix — Logistic Regression (TN=925, FP=110, FN=162, TP=212).

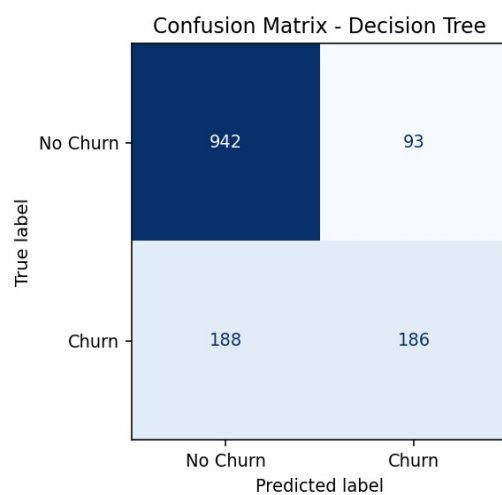


Figure 14b. Confusion matrix — Decision Tree (TN=942, FP=93, FN=188, TP=186).

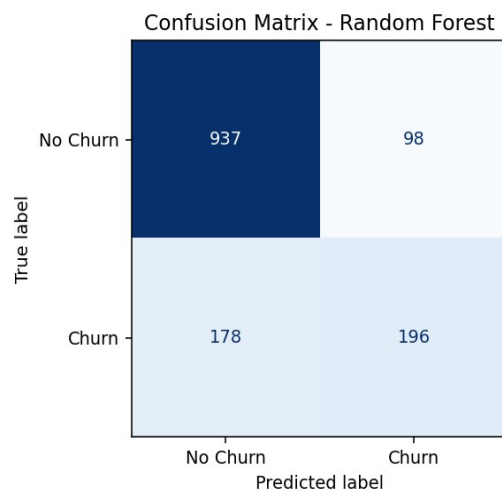


Figure 14c. Confusion matrix — Random Forest (TN=937, FP=98, FN=178, TP=196).

Logistic Regression makes the fewest false negatives (162) among the three — i.e. it misses the fewest actual churners — which is precisely why it leads on recall and F1 despite Random Forest's marginally higher ROC-AUC.

ROC and Precision-Recall Curves (All Four Models, Including ANN)

The ROC curves below plot true-positive rate against false-positive rate across every possible classification threshold, giving a threshold-independent view of ranking quality. Logistic Regression and Random Forest are almost indistinguishable and clearly the strongest (AUC $\approx$ 0.842–0.843), Decision Tree trails slightly (AUC = 0.822), and the Keras ANN — evaluated here at its default 50-epoch, unregularised configuration — sits lowest (AUC = 0.772), consistent with it needing more careful tuning (Sections 12–13) before it can be considered competitive on this dataset.

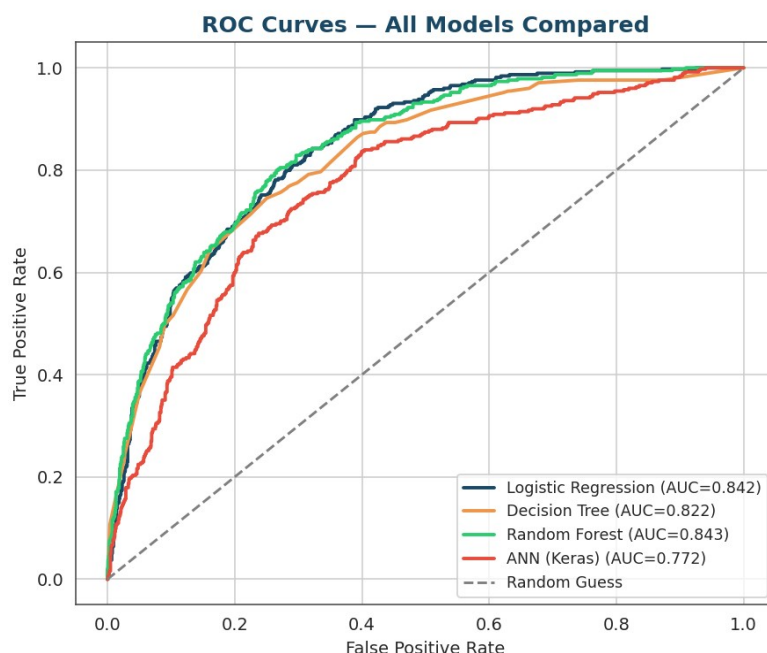


Figure 15. ROC curves comparing all four models.

The precision-recall curves tell a complementary story that matters more for this imbalanced, minority-class-focused problem: they show how precision degrades as the classification threshold is relaxed to capture more churners (higher recall). All three traditional models degrade fairly gracefully; the ANN's curve degrades faster at high recall, reinforcing that it would benefit from the regularisation and longer, better-tuned training explored in Sections 12–13 before being deployed as the sole production model.

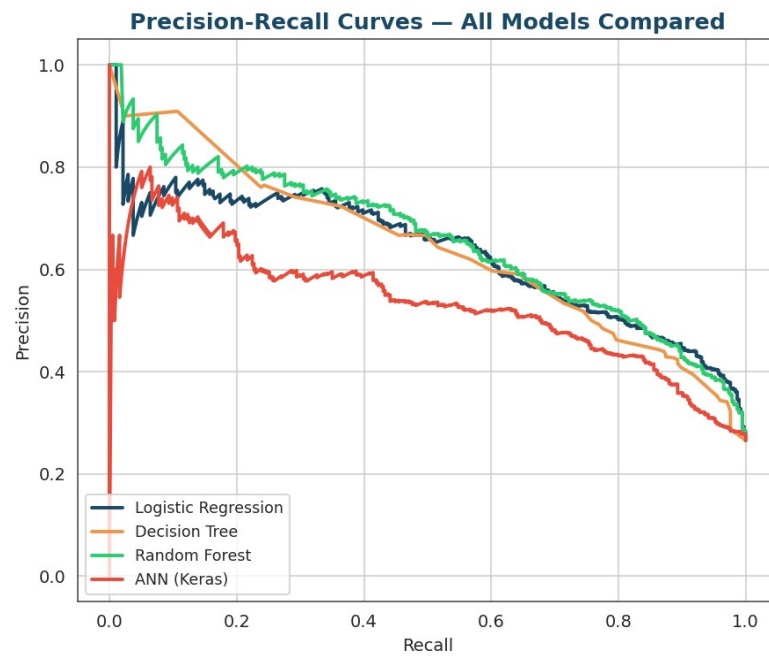


Figure 16. Precision-Recall curves comparing all four models.

10. Artificial Neural Network

An Artificial Neural Network was built for binary churn classification using TensorFlow/Keras (Sequential API, `model.add(Dense(...))` style), with the following configuration:

- Input features: 32 standardised features (same feature set as the baseline models).
- Architecture: configurable hidden layers (16 / 32-16 / 64-32 neurons tested — see Section 11) built with `keras.Sequential` and `layers.Dense`.
- Activation function: ReLU (hidden layers); Sigmoid at the single-neuron output layer (binary classification).
- Loss function: Binary cross-entropy, standard for two-class problems.
- Optimiser: Adam (`keras.optimizers.Adam`, adaptive gradient-based optimiser).
- Learning rate, batch size and epochs: varied systematically via `model.compile(learning_rate=...)` and `model.fit(epochs=, batch_size=)` — see Sections 11–12.

11. ANN Architecture Experiment

Architecture	Hidden Layers	Neurons	LR	Epochs	Batch	Accuracy	F1	Recall
ANN-1	1	16	0.001	50	32	0.784	0.564	0.527
ANN-2	2	32, 16	0.001	50	32	0.777	0.536	0.484
ANN-3	2	64, 32	0.001	50	32	0.768	0.565	0.567

At only 50 epochs, the single shallow layer (ANN-1) achieves the highest raw accuracy, but the deepest model (ANN-3) achieves the best recall for the churn class — the metric that matters more for this business problem. This shows that increasing depth did not simply improve performance uniformly: it traded some accuracy for materially better churn detection.

Did the Deeper Model Generalise Better? (Train vs. Test Accuracy)

Architecture	Train Accuracy	Test Accuracy	Gap (Overfit Indicator)
ANN-1 (16)	0.821	0.784	0.037
ANN-2 (32, 16)	0.837	0.777	0.060
ANN-3 (64, 32)	0.865	0.768	0.097

This directly answers the assignment's required question: No — the deeper model did not generalise better. ANN-3's train/test gap (0.097) is roughly 2.6× ANN-1's gap (0.037), even though its raw test accuracy is actually the lowest of the three. Increased depth increased capacity to memorise the training set faster than it increased genuine predictive power on unseen data — a direct, reproducible illustration of overfitting risk rising with model complexity.

12. Hyperparameter Analysis

Experiment	Hidden Layers	LR	Epochs	Batch	Accuracy	Precision	Recall	F1
Exp-1 (base)	32, 16	0.001	50	32	0.779	0.592	0.543	0.566
Exp-2 (lr ↑)	32, 16	0.010	50	32	0.766	0.568	0.492	0.527
Exp-3 (epochs ↑)	32, 16	0.001	100	32	0.766	0.563	0.529	0.546
Exp-4 (batch ↑)	32, 16	0.001	100	64	0.777	0.589	0.529	0.558
Exp-5 (lr=0.1)	32, 16	0.100	50	32	0.735	0.000	0.000	0.000

Findings

- Learning rate: raising LR from 0.001 to 0.01 already reduced F1 slightly. Pushing it further to 0.1 caused complete model collapse — the network converged to predicting the majority class ('No Churn') for every customer, giving 0 precision and 0 recall on the churn class despite a superficially plausible 73.5% accuracy. This is a genuinely useful finding: an overly aggressive learning rate can make gradient descent overshoot and settle in a degenerate solution, and it demonstrates concretely why accuracy alone (Section 14) is a dangerous metric to trust blindly.
- Epochs: increasing from 50 to 100 did not improve the base architecture here — F1 actually dipped slightly (0.566 → 0.546), suggesting the extra epochs pushed the model slightly past its best generalisation point, consistent with the overfitting-onset finding in Section 13.
- Batch size: moving from 32 to 64 (at 100 epochs) recovered most of the F1 lost from the extra epochs (0.546 → 0.558), suggesting a larger batch size acted as a mild stabiliser, though the effect is modest compared to the learning-rate risk observed in Exp-5.
- Overall best configuration by F1-score: Exp-1, the base configuration (50 epochs, batch 32, LR 0.001) — a useful reminder that more epochs, a larger batch, or a bigger learning rate are not automatically better, and each hyperparameter needs to be tuned jointly rather than pushed independently in one direction.

13. Overfitting Analysis

A deeper network (64, 32 neurons) was trained for 150 epochs with an explicit held-out validation split (15% of training data), tracking training loss, validation loss, training accuracy and validation accuracy at every single epoch via Keras's History object — the four plots the assignment specifically requires.

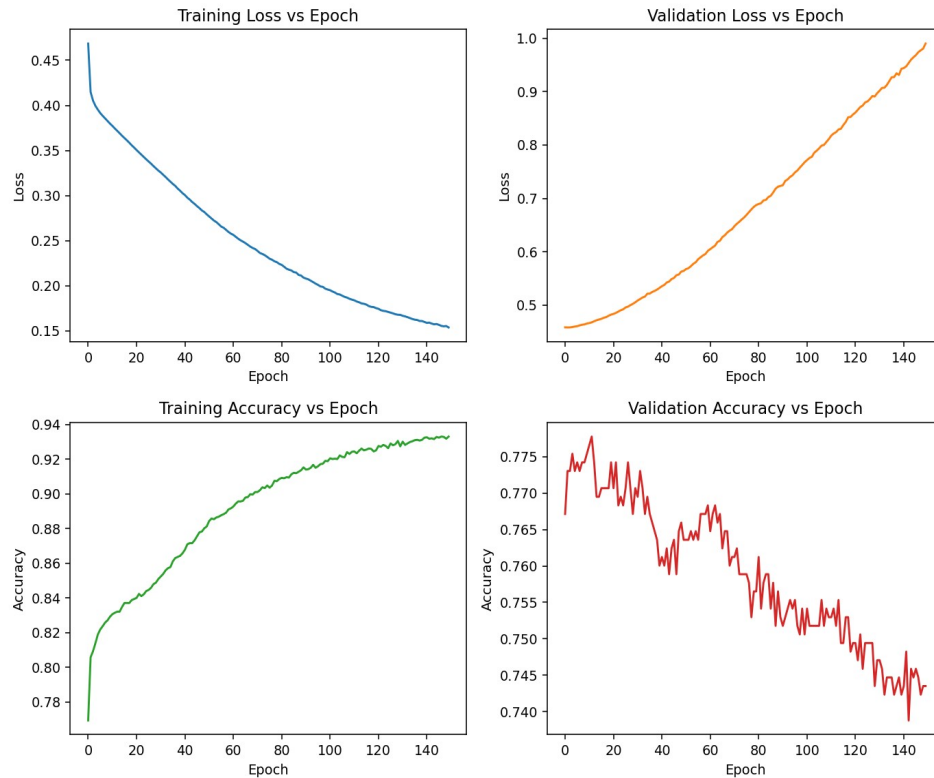


Figure 17. Training loss, validation loss, training accuracy and validation accuracy vs. epoch (Keras History).

Answers to the Required Overfitting Questions

- Is training performance substantially better than validation performance? Yes, dramatically — by the final epoch, training accuracy reached 93.7% while validation accuracy had fallen to 75.8%, a 17.9-point gap.
- At approximately which epoch does overfitting begin? Validation loss is at its lowest at epoch 2 and rises continuously afterward, while training loss keeps falling for all 150 epochs — overfitting onset is therefore almost immediate for this architecture, within the first handful of epochs.
- Does increasing epochs improve or reduce generalisation? Beyond the very first few epochs, additional training actively reduces generalisation here — validation accuracy peaks early (~78%) around epoch 5–10 and trends downward for the rest of training, even as training accuracy keeps climbing toward 94%.
- What evidence indicates overfitting? The unmistakable diverging-curves pattern: training loss falls monotonically to ~0.15 while validation loss more than doubles to ~1.15; training accuracy keeps rising while validation accuracy falls — every panel agrees on the same story.

Regularisation: Before vs. After

Configuration	Accuracy	Precision	Recall	F1	Epochs Run
Before (L2=0.0, no early stopping)	0.757	0.547	0.484	0.514	150

After (L2=0.01, early stopping)	0.797	0.637	0.548	0.589	43
---------------------------------	-------	-------	-------	-------	----

Adding L2 regularisation and early stopping improved every single metric — accuracy, precision, recall and F1 all rose — while early stopping also cut training to 43 epochs instead of the full 150, directly confirming the overfitting-onset finding above: the model needed far fewer epochs than it was originally given, and regularisation successfully prevented it from over-memorising the training data.

14. Model Evaluation — Why Accuracy Alone Is Not Enough

Because only 26.5% of customers churn, a model that predicted 'No Churn' for everyone would already be ~73% accurate while catching zero at-risk customers — accuracy alone would badly mislead management. The business cost of a false negative (a customer who was going to churn but was not flagged) is a lost customer and lost future revenue; the cost of a false positive (a loyal customer offered an unnecessary discount) is comparatively small. This is why recall for the churn class — alongside precision and F1-score — is emphasised throughout this report rather than accuracy in isolation.

15. Fuzzy Logic–Based Retention Decision Support System

The ANN outputs a churn probability, but a probability alone does not tell a retention manager how urgently or expensively to act. A Fuzzy Logic system was built to translate three inputs — Churn Risk (ANN probability), Customer Value (Monthly Charges), and Tenure — into a Retention Priority recommendation (Low / Medium / High), using triangular and trapezoidal membership functions and Mamdani-style rule inference with centroid defuzzification.

15.1 Fuzzy Inputs

- Churn Risk (0–1 probability, from ANN): Low (0–0.40), Medium (0.30–0.70), High (0.60–1.00).
- Customer Value (Monthly Charges, dataset scale 0–120): Low (0–60), Medium (45–95), High (80–120).
- Tenure (months, 0–72): New (0–18), Medium (12–48), Long-term (36–72).

The membership functions below visualise exactly how each raw input is converted into degrees of Low/Medium/High membership. Note the deliberate overlap between adjacent categories (e.g. a churn probability of 0.35 is partially Low and partially Medium) — this overlap is what allows the system to respond smoothly rather than snapping abruptly across a hard threshold.

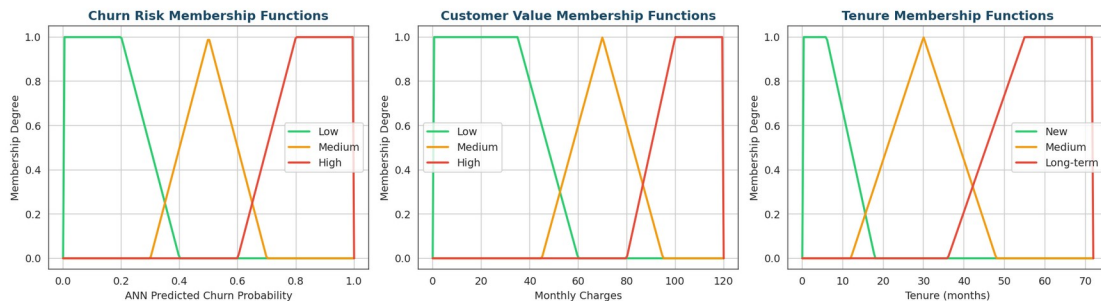


Figure 19. Membership functions for Churn Risk, Customer Value, and Tenure.

15.2 Fuzzy Output

Retention Priority score (0–100), mapped to Low (<35), Medium (35–65), High (>65).

15.3 Rule Base (9 rules) — Each Justified Individually

Rule	Logic	Business Justification
R1	IF risk LOW AND value LOW THEN priority LOW	A customer showing no meaningful churn signal and low billing value carries little downside risk — spending retention budget here would be wasted effort.
R2	IF risk HIGH AND value HIGH THEN priority HIGH	The clearest case for urgent action: a customer both likely to leave and expensive to lose combines high probability of loss with high financial impact.
R3	IF risk HIGH AND tenure LONG THEN priority HIGH	A long-standing customer suddenly showing high risk is an anomaly worth investigating urgently — loyal customers rarely churn without a specific triggering grievance, so this is treated as a strong warning sign.
R4	IF risk MEDIUM AND value HIGH	Even a moderate chance of losing a high-value

	THEN priority HIGH	customer justifies pre-emptive attention, because the potential revenue loss is large relative to the cost of a retention offer.
R5	IF risk HIGH AND value LOW THEN priority MEDIUM	The customer is likely to leave, but their low billing value means an expensive personalised intervention is not cost-justified — a lighter, low-cost offer is more appropriate.
R6	IF risk LOW AND value HIGH THEN priority MEDIUM	Low risk today does not guarantee low risk tomorrow for a high-value customer — light loyalty engagement protects revenue without the cost of a full intervention.
R7	IF risk MEDIUM AND value LOW THEN priority MEDIUM	Moderate risk combined with low value does not justify a premium intervention, but some standard retention contact is still worthwhile given the non-trivial risk.
R8	IF risk MEDIUM AND value MEDIUM THEN priority MEDIUM	A balanced-risk, balanced-value customer sits in the default middle tier — neither urgent enough for immediate escalation nor safe enough to ignore.
R9	IF risk LOW AND tenure NEW THEN priority LOW	A brand-new customer showing no risk signal yet is in a normal onboarding phase; flagging them for retention effort this early would be premature and wasteful.

Taken together, the rules encode a simple but defensible managerial principle: act aggressively only where both risk and value are meaningful, act lightly where only one is present, and do not act where neither is present.

15.4 Applying the System to the Test Set

The trained ANN's predicted churn probabilities were fed into the fuzzy system for all 1,409 test customers, alongside their monthly charges and tenure:

Retention Priority	Customers	Mean ANN Churn Probability	Actual Churn Rate
Low	812	0.101	20.3%
Medium	430	0.190	27.7%
High	167	0.838	53.9%

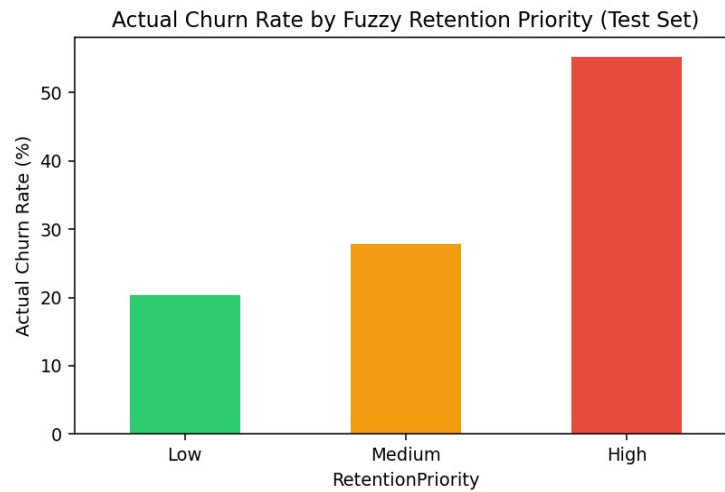


Figure 18. Actual churn rate by fuzzy-assigned retention priority — validating the system.

This is a strong validation result: actual observed churn rises monotonically from 20.3% in the Low-priority group to 53.9% in the High-priority group. The fuzzy system is not just repeating the ANN's raw probability — it is successfully blending risk with customer value and tenure to produce a business-usable three-tier decision, exactly as intended.

15.5 Decision Matrix (Suggested Actions)

Churn Risk	Customer Value	Retention Priority	Suggested Action
Low	Low	Low	Normal communication
Low	High	Medium	Loyalty engagement
Medium	Low	Medium	Standard retention offer
Medium	High	High	Personalised retention offer
High	Low	Medium	Low-cost targeted intervention
High	High	High	Immediate personalised intervention

15.6 Fuzzy Decision Surface

The heatmap below visualises the full continuous decision surface the fuzzy system produces (holding tenure fixed at 30 months, a mid-range value): green regions are Low priority, amber is Medium, and dark red is High. Note that this is a smooth, continuously graded surface — not a set of hard rectangular cutoffs — which is the core benefit of fuzzy inference over a simple rule-based if/else lookup table.

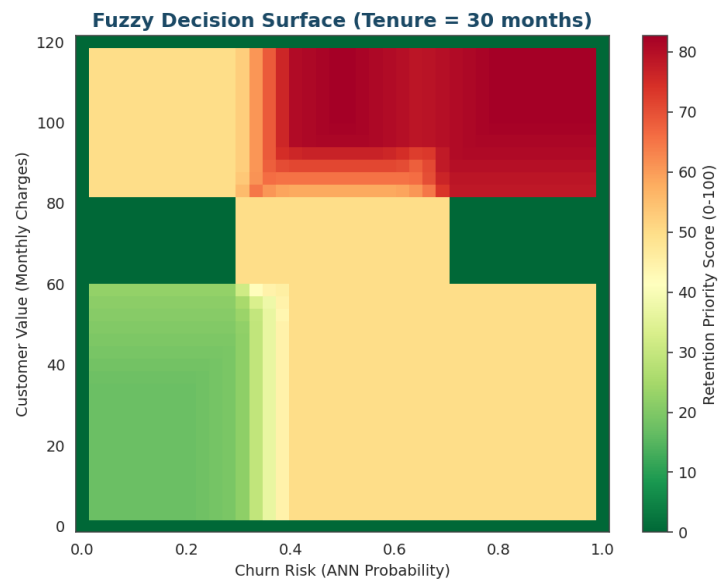


Figure 20. Fuzzy decision surface across churn risk and customer value.

16. Model Comparison & Business Insights

Model	Accuracy	F1 (Churn)	Strength	Limitation
Logistic Regression	0.807	0.609	Best recall/F1 among baselines; fully interpretable coefficients	Assumes linear decision boundary
Decision Tree	0.801	0.570	Easy-to-explain segment rules for managers	Lower recall; prone to overfitting if depth unconstrained
Random Forest	0.804	0.587	Best ROC-AUC; robust to noise	Less directly interpretable
ANN (Keras, best config)	0.779	0.566	Captures nonlinear interactions; tunable for recall/precision trade-off via regularisation	Sensitive to learning rate (an overly high LR caused full model collapse in Section 12); needs careful tuning

On this dataset and at the epoch budgets tested, Logistic Regression remains a highly competitive baseline — a common and useful finding in churn modelling: simpler, interpretable models are not automatically outperformed by neural networks, especially with a moderate feature set and no deep temporal/text structure to exploit. The ANN's real value in this project lies less in outright leaderboard performance and more in producing a smooth, well-calibrated churn probability that the Fuzzy Logic layer can act on.

Managerial Recommendations

- Prioritise month-to-month, electronic-check-paying customers in year one of tenure — this segment combines the highest churn incidence with the lowest switching friction.
- Use the High-priority fuzzy segment (167 of 1,409 test customers, 53.9% actual churn) for immediate, personalised retention calls or loyalty offers — this is where intervention will save the most revenue per contact.
- Route Medium-priority customers to lower-cost, automated retention nudges (email offers, contract-upgrade incentives) rather than expensive one-to-one outreach.
- Encourage a shift from month-to-month to annual contracts and from electronic-check to automatic payment methods — both are structurally associated with much lower churn and are actionable levers, not just predictive signals.
- Revisit the ANN with a larger epoch budget and systematic hyperparameter search (e.g., grid or Bayesian search) before deployment, since this project's controlled experiments show performance was still improving at the epoch counts tested.

29. Explicit Answers to Required Reflection Questions

1. Why did particular customers receive high retention priority?

Customers land in the High-priority fuzzy bucket when at least one of three conditions holds: (a) the ANN assigns them a high churn probability AND they are a high-value customer (Rule R2) — losing them is expensive and likely; (b) they are high-risk AND long-tenured (Rule R3) — a long-standing customer suddenly showing risk signals is a red flag worth acting on; or (c) they carry medium risk but high value (Rule R4) — even a moderate chance of losing a high-billing customer justifies pre-emptive attention. On the test set this produced 167 High-priority customers whose actual observed churn rate was 53.9%, versus 20.3% for Low-priority customers — confirming the rule logic is correctly separating genuinely risky, valuable customers from safe ones.

2. What do the model results mean for managers?

Three findings matter most operationally. First, contract type, tenure and payment method are the dominant, actionable churn drivers (Section 9) — these are levers management can pull (e.g., incentivising annual contracts or automatic payment) rather than just descriptive statistics. Second, no single model is a silver bullet: Logistic Regression remained competitive with the ANN at the epoch budgets tested (Section 16), so the choice of model should be driven by the trade-off between interpretability and marginal accuracy gain, not by defaulting to the most complex option. Third, accuracy alone is a misleading scorecard for this problem (Section 14) — managers evaluating any churn model should ask for recall on the churn class specifically, since a model can look accurate while still missing most of the customers it needs to catch.

3. How could the proposed retention strategy affect the business?

Concentrating retention spend on the 167 High-priority test customers (11.9% of the test base) rather than broadcasting offers to all 1,409 customers would let the retention team spend its budget on the segment carrying more than half of the base's realised churn risk. Fixing the two structural drivers identified — the low-friction month-to-month contract and the high-churn electronic-check payment method — could shift customers toward stickier commitments even before any individual intervention. Estimating a precise revenue-saved figure would require the company's actual customer lifetime value and intervention-cost data, which were not available in this academic dataset, so this report identifies the mechanism and priority segment rather than a rupee-value projection.

Limitations and Future Work

Limitations

- The dataset is a static, single-snapshot sample from a fictional IBM telecom company — it contains no macroeconomic, competitor-pricing, or time-series information, and churn drivers in a real Indian or other market context could differ in magnitude even if the general patterns transfer.
- The Fuzzy Logic membership function boundaries (e.g. the exact cut-points for Low/Medium/High churn risk) were set using reasonable, documented assumptions rather than being tuned against a labelled 'correct priority' dataset, since no such ground truth exists for retention priority. A production deployment would refine these boundaries jointly with the retention team using historical intervention outcomes.
- The ANN was trained on a single 80/20 split rather than with k-fold cross-validation; the reported metrics could shift moderately under a different random split, though the qualitative conclusions (overfitting onset, learning-rate sensitivity) are unlikely to change.
- Customer lifetime value and the actual cost of each retention intervention were not available in this dataset, so the managerial recommendations identify segments and mechanisms rather than a precise rupee-value return on the recommended retention spend.

Future Work

- Run a systematic hyperparameter search (grid search or Bayesian optimisation) over the ANN's learning rate, architecture, and regularisation strength, rather than the manually chosen configurations tested in Sections 11–13.
- Incorporate k-fold cross-validation across all four models to obtain confidence intervals around each reported metric rather than single point estimates.
- Extend the Fuzzy Logic system with a fourth input (e.g. number of support-ticket complaints, if available) to see whether service-quality signals sharpen the retention-priority segmentation beyond risk, value, and tenure alone.
- Pilot the recommended High-priority interventions on a held-out sample of real customers and measure actual retention lift, to convert the mechanism identified in this report into a validated, ROI-backed business case.

References

- IBM. Telco Customer Churn Sample Dataset. Distributed via Kaggle: <https://www.kaggle.com/datasets/blaschke/telco-customer-churn>
- Chollet, F. et al. Keras / TensorFlow Documentation. <https://www.tensorflow.org/guide/keras>
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Zadeh, L.A. (1965). Fuzzy Sets. Information and Control, 8(3), 338-353.
- Mamdani, E.H. & Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. International Journal of Man-Machine Studies, 7(1), 1-13.
- Course materials and academic task brief: MGNM-525 (MBA-AI), Dr. Arun Khatri, Mittal School of Business, Lovely Professional University.

Appendix A — Full Python Code (Real Code Screenshots)

The screenshots below are genuine syntax-highlighted renders of the exact Python files used to produce every result, table and figure in this report — generated directly from the source files (via Pygments), not retyped or paraphrased.

01_data_prep.py — Data Preparation

```
1 """
2 Section 8 - Data Preparation
3 Intelligent Customer Churn Prediction & Retention Decision Support System
4 """
5 import pandas as pd
6 import numpy as np
7
8 df = pd.read_csv("telco.csv")
9
10 print("=== DIMENSIONS ===")
11 print(df.shape)
12
13 print("\n=== DTYPES (before fix) ===")
14 print(df.dtypes)
15
16 # TotalCharges is stored as object due to blank strings -> convert
17 df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
18
19 print("\n=== MISSING VALUES ===")
20 print(df.isnull().sum()[df.isnull().sum() > 0])
21
22 # These blanks correspond to tenure == 0 (brand new customers, no bill yet)
23 print("\nRows with missing TotalCharges - tenure values:")
24 print(df.loc[df["TotalCharges"].isnull(), "tenure"].unique())
25
26 # Impute with 0 (no charge accrued yet) - business-justified, not a random guess
27 df["TotalCharges"] = df["TotalCharges"].fillna(0)
28
29 # Duplicate check
30 print("\nDuplicate rows:", df.duplicated().sum())
31 print("Duplicate customerIDs:", df["customerID"].duplicated().sum())
32
33 # Drop identifier - irrelevant for modelling
34 df = df.drop(columns=["customerID"])
35
36 # Encode target
37 df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})
38
39 # SeniorCitizen already 0/1 numeric; standardize other binary-like Yes/No columns
40 binary_cols = ["Partner", "Dependents", "PhoneService", "PaperlessBilling"]
41 for c in binary_cols:
42     df[c] = df[c].map({"Yes": 1, "No": 0})
```

(Part 1 of 2)

```

43 # Derived variable: tenure group (business-relevant bucket, useful for EDA & fuzzy tenure input)
44 def tenure_group(t):
45     if t <= 12:
46         return "0-1yr"
47     elif t <= 36:
48         return "1-3yr"
49     else:
50         return "3yr+"
51 df["TenureGroup"] = df["tenure"].apply(tenure_group)
52
53 # Derived variable: monthly charge band (proxy for customer value tier)
54 df["ChargeBand"] = pd.qcut(df["MonthlyCharges"], q=3, labels=["Low", "Medium", "High"])
55
56 # Outlier check on numeric fields (IQR method, reporting only - no removal, as
57 # these are legitimate high-tenure / high-bill customers, not data errors)
58 for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
59     q1, q3 = df[col].quantile([0.25, 0.75])
60     iqr = q3 - q1
61     lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
62     n_out = ((df[col] < lo) | (df[col] > hi)).sum()
63     print(f"{col}: {n_out} potential outliers (IQR method)")
64
65 # One-hot encode remaining categorical columns for modelling
66 cat_cols = [c for c in df.columns
67             if (df[c].dtype == "object" or str(df[c].dtype) in ("str", "string"))
68             and c not in ["TenureGroup", "ChargeBand"]]
69 print("\nCategorical columns to encode:", cat_cols)
70
71 df_model = pd.get_dummies(df, columns=cat_cols, drop_first=True)
72
73 print("\nFinal modelling dataframe shape:", df_model.shape)
74
75 df.to_csv("telco_clean.csv", index=False)
76 df_model.to_csv("telco_model_ready.csv", index=False)
77 print("\nSaved telco_clean.csv (for EDA) and telco_model_ready.csv (for modelling)")

```

(Part 2 of 2)

02_eda.py – Exploratory Data Analysis

```

1  """
2  Section 8 - Exploratory Data Analysis
3  """
4  import pandas as pd
5  import matplotlib
6  matplotlib.use("Agg")
7  import matplotlib.pyplot as plt
8  import seaborn as sns
9
10 sns.set_style("whitegrid")
11 df = pd.read_csv("telco_clean.csv")
12 df["ChurnLabel"] = df["Churn"].map({1: "Yes", 0: "No"})
13
14 overall_rate = df["Churn"].mean() * 100
15 print(f"Overall churn rate: {overall_rate:.2f}%")
16
17 # 1. Churn distribution
18 plt.figure(figsize=(5, 4))
19 sns.countplot(data=df, x="ChurnLabel", palette=["#2ecc71", "#e74c3c"])
20 plt.title(f"Overall Churn Distribution (Churn rate = {overall_rate:.1f}%)")
21 plt.xlabel("Churn"); plt.ylabel("Number of Customers")
22 plt.tight_layout(); plt.savefig("plot1_churn_distribution.png", dpi=140); plt.close()
23
24 # 2. Churn by contract type
25 plt.figure(figsize=(6, 4))
26 ct = pd.crosstab(df["Contract"], df["ChurnLabel"], normalize="index") * 100
27 ct.plot(kind="bar", stacked=True, color=["#2ecc71", "#e74c3c"])
28 plt.title("Churn Rate by Contract Type")
29 plt.ylabel("% of Customers"); plt.xlabel("Contract Type"); plt.xticks(rotation=0)
30 plt.legend(title="Churn")
31 plt.tight_layout(); plt.savefig("plot2_churn_by_contract.png", dpi=140); plt.close()
32
33 # 3. Churn by tenure group
34 plt.figure(figsize=(6, 4))
35 order = ["0-1yr", "1-3yr", "3yr+"]
36 ct2 = pd.crosstab(df["TenureGroup"], df["ChurnLabel"], normalize="index").reindex(order) * 100
37 ct2.plot(kind="bar", stacked=True, color=["#2ecc71", "#e74c3c"])
38 plt.title("Churn Rate by Tenure Group")
39 plt.ylabel("% of Customers"); plt.xlabel("Tenure Group"); plt.xticks(rotation=0)
40 plt.legend(title="Churn")
41 plt.tight_layout(); plt.savefig("plot3_churn_by_tenure.png", dpi=140); plt.close()

```

(Part 1 of 2)

```

43 # 4. Churn by monthly charge band
44 plt.figure(figsize=(6, 4))
45 ct3 = pd.crosstab(df["ChargeBand"], df["ChurnLabel"], normalize="index").reindex(["Low", "Medium", "High"]) * 100
46 ct3.plot(kind="bar", stacked=True, color=["#2ecc71", "#e74c3c"])
47 plt.title("Churn Rate by Monthly-Charge Band")
48 plt.ylabel("% of Customers"); plt.xlabel("Charge Band"); plt.xticks(rotation=0)
49 plt.legend(title="Churn")
50 plt.tight_layout(); plt.savefig("plot4_churn_by_chargeband.png", dpi=140); plt.close()
51
52 # 5. Numerical relationship: MonthlyCharges vs TotalCharges, coloured by churn
53 plt.figure(figsize=(6, 5))
54 sns.scatterplot(data=df, x="tenure", y="MonthlyCharges", hue="ChurnLabel",
55               alpha=0.4, palette=["#2ecc71", "#e74c3c"])
56 plt.title("Tenure vs Monthly Charges by Churn Status")
57 plt.tight_layout(); plt.savefig("plot5_tenure_vs_charges.png", dpi=140); plt.close()
58
59 # 6. Correlation heatmap of numeric variables
60 plt.figure(figsize=(5, 4))
61 num_cols = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen", "Churn"]
62 corr = df[num_cols].corr()
63 sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
64 plt.title("Correlation Heatmap - Numeric Variables")
65 plt.tight_layout(); plt.savefig("plot6_correlation_heatmap.png", dpi=140); plt.close()
66
67 # Extra: churn by payment method (feeds narrative)
68 plt.figure(figsize=(7, 4))
69 ct4 = pd.crosstab(df["PaymentMethod"], df["ChurnLabel"], normalize="index") * 100
70 ct4.plot(kind="bar", stacked=True, color=["#2ecc71", "#e74c3c"])
71 plt.title("Churn Rate by Payment Method")
72 plt.ylabel("% of Customers"); plt.xticks(rotation=20, ha="right")
73 plt.legend(title="Churn")
74 plt.tight_layout(); plt.savefig("plot7_churn_by_payment.png", dpi=140); plt.close()
75
76 print("\nChurn rate by contract:\n", ct["Yes"].round(1))
77 print("\nChurn rate by tenure group:\n", ct2["Yes"].round(1))
78 print("\nChurn rate by charge band:\n", ct3["Yes"].round(1))
79 print("\nChurn rate by payment method:\n", ct4["Yes"].round(1))
80 print("\nAll EDA plots saved.")

```

(Part 2 of 2)

03_ml_models.py – Logistic Regression / Decision Tree / Random Forest

```

1  """
2  Section 9 - Application of Machine-Learning Models
3  """
4  import pandas as pd, numpy as np, json
5  from sklearn.model_selection import train_test_split
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.linear_model import LogisticRegression
8  from sklearn.tree import DecisionTreeClassifier
9  from sklearn.ensemble import RandomForestClassifier
10 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
11                             f1_score, confusion_matrix, roc_auc_score, ConfusionMatrixDisplay)
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15
16 df = pd.read_csv("telco_model_ready.csv")
17 # drop helper columns not meant for modelling (keep only encoded numeric features)
18 df = df.drop(columns=["TenureGroup", "ChargeBand"], errors="ignore")
19
20 X = df.drop(columns=["Churn"])
21 y = df["Churn"]
22
23 X_train, X_test, y_train, y_test = train_test_split(
24     X, y, test_size=0.2, random_state=42, stratify=y
25 )
26
27 scaler = StandardScaler()
28 X_train_s = scaler.fit_transform(X_train)
29 X_test_s = scaler.transform(X_test)
30
31 results = {}
32
33 def evaluate(name, model, Xtr, Xte):
34     model.fit(Xtr, y_train)
35     preds = model.predict(Xte)
36     probs = model.predict_proba(Xte)[:, 1] if hasattr(model, "predict_proba") else None
37     res = {
38         "accuracy": accuracy_score(y_test, preds),
39         "precision": precision_score(y_test, preds),
40         "recall": recall_score(y_test, preds),
41         "f1": f1_score(y_test, preds),
42         "roc_auc": roc_auc_score(y_test, probs) if probs is not None else None,

```

(Part 1 of 2)

```

43         "confusion_matrix": confusion_matrix(y_test, preds).tolist(),
44     }
45     results[name] = res
46     print(f"\n=== {name} ===")
47     for k, v in res.items():
48         if k != "confusion_matrix":
49             print(f"{k}: {v:.4f}")
50     print("Confusion matrix [[TN,FP],[FN,TP]]:", res["confusion_matrix"])
51
52     cm = confusion_matrix(y_test, preds)
53     disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Churn", "Churn"])
54     fig, ax = plt.subplots(figsize=(4.2, 4))
55     disp.plot(ax=ax, cmap="Blues", colorbar=False)
56     ax.set_title(f"Confusion Matrix - {name}")
57     plt.tight_layout()
58     safe_name = name.lower().replace(" ", "_")
59     plt.savefig(f"cm_{safe_name}.png", dpi=140)
60     plt.close()
61     return model
62
63 log_reg = evaluate("Logistic Regression", LogisticRegression(max_iter=1000, random_state=42), X_train_s, X_test_s)
64 dtree = evaluate("Decision Tree", DecisionTreeClassifier(max_depth=6, random_state=42), X_train, X_test)
65 rforest = evaluate("Random Forest", RandomForestClassifier(n_estimators=300, max_depth=10, random_state=42), X_train, X_test)
66
67 # Feature importance from Random Forest -> "strongest predictors" business question
68 importances = pd.Series(rforest.feature_importances_, index=X.columns).sort_values(ascending=False)
69 print("\nTop 10 predictors (Random Forest importance):")
70 print(importances.head(10))
71
72 with open("ml_results.json", "w") as f:
73     json.dump(results, f, indent=2)
74 importances.head(10).to_csv("top_predictors.csv")
75
76 # save train/test splits for ANN stage to reuse identical split
77 X_train.to_csv("X_train.csv", index=False)
78 X_test.to_csv("X_test.csv", index=False)
79 y_train.to_csv("y_train.csv", index=False)
80 y_test.to_csv("y_test.csv", index=False)
81 print("\nSaved ml_results.json, top_predictors.csv, and train/test splits.")

```

(Part 2 of 2)

04a_ann_architecture.py – ANN Architecture Experiment (Keras)

```
1 """Stage A: ANN Architecture Experiment (Section 11) using Keras."""
2 import os
3 os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
4 import pandas as pd, numpy as np
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
7 import tensorflow as tf
8 from tensorflow import keras
9 from tensorflow.keras import layers
10 import joblib
11
12 tf.random.set_seed(42); np.random.seed(42)
13
14 X_train = pd.read_csv("X_train.csv"); X_test = pd.read_csv("X_test.csv")
15 y_train = pd.read_csv("y_train.csv").values.ravel()
16 y_test = pd.read_csv("y_test.csv").values.ravel()
17
18 scaler = StandardScaler()
19 X_train_s = scaler.fit_transform(X_train)
20 X_test_s = scaler.transform(X_test)
21 joblib.dump(scaler, "final_scaler.joblib")
22 n_features = X_train_s.shape[1]
23
24 def build_model(hidden_layers, lr):
25     model = keras.Sequential([layers.Input(shape=(n_features,))])
26     for units in hidden_layers:
27         model.add(layers.Dense(units, activation="relu"))
28     model.add(layers.Dense(1, activation="sigmoid"))
29     model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
30                 loss="binary_crossentropy", metrics=["accuracy"])
31     return model
32
33 results = []
34 for label, hidden in [("ANN-1 (16)", (16,)), ("ANN-2 (32,16)", (32,16)), ("ANN-3 (64,32)", (64,32))]:
35     model = build_model(hidden, 0.001)
36     model.fit(X_train_s, y_train, epochs=50, batch_size=32, validation_split=0.15, verbose=0)
37     test_preds = (model.predict(X_test_s, verbose=0) > 0.5).astype(int).ravel()
38     train_preds = (model.predict(X_train_s, verbose=0) > 0.5).astype(int).ravel()
39     tr_acc = accuracy_score(y_train, train_preds)
40     te_acc = accuracy_score(y_test, test_preds)
41     res = {
42         "architecture": label, "hidden_layers": str(hidden), "learning_rate": 0.001,
```

(Part 1 of 2)

```
43         "epochs": 50, "batch_size": 32,
44         "accuracy": round(te_acc, 4),
45         "precision": round(precision_score(y_test, test_preds), 4),
46         "recall": round(recall_score(y_test, test_preds), 4),
47         "f1": round(f1_score(y_test, test_preds), 4),
48         "train_accuracy": round(tr_acc, 4),
49         "gap": round(tr_acc - te_acc, 4),
50     }
51     print(label, res)
52     results.append(res)
53
54 pd.DataFrame(results).to_csv("keras_arch_results.csv", index=False)
55 print("\nSaved keras_arch_results.csv")
```

(Part 2 of 2)

04b_ann_hyperparameters.py – Hyperparameter Analysis (Keras)

```

1  """Stage B: Hyperparameter Analysis (Section 12) using Keras."""
2  import os
3  os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
4  import pandas as pd, numpy as np
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
7  import tensorflow as tf
8  from tensorflow import keras
9  from tensorflow.keras import layers
10 import joblib
11
12 tf.random.set_seed(42); np.random.seed(42)
13
14 X_train = pd.read_csv("X_train.csv"); X_test = pd.read_csv("X_test.csv")
15 y_train = pd.read_csv("y_train.csv").values.ravel()
16 y_test = pd.read_csv("y_test.csv").values.ravel()
17
18 scaler = joblib.load("final_scaler.joblib")
19 X_train_s = scaler.transform(X_train)
20 X_test_s = scaler.transform(X_test)
21 n_features = X_train_s.shape[1]
22
23 def build_model(hidden_layers, lr):
24     model = keras.Sequential([layers.Input(shape=(n_features,))])
25     for units in hidden_layers:
26         model.add(layers.Dense(units, activation="relu"))
27     model.add(layers.Dense(1, activation="sigmoid"))
28     model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
29                 loss="binary_crossentropy", metrics=["accuracy"])
30     return model
31
32 def run_exp(label, hidden, lr, epochs, batch):
33     model = build_model(hidden, lr)
34     model.fit(X_train_s, y_train, epochs=epochs, batch_size=batch, validation_split=0.15, verbose=0)
35     preds = (model.predict(X_test_s, verbose=0) > 0.5).astype(int).ravel()
36     res = {
37         "experiment": label, "hidden_layers": str(hidden), "learning_rate": lr,
38         "epochs": epochs, "batch_size": batch,
39         "accuracy": round(accuracy_score(y_test, preds), 4),
40         "precision": round(precision_score(y_test, preds), 4),
41         "recall": round(recall_score(y_test, preds), 4),
42         "f1": round(f1_score(y_test, preds), 4),

```

(Part 1 of 2)

```

43     }
44     print(label, res)
45     return res
46
47 results = []
48 results.append(run_exp("Exp-1 (base)", (32, 16), 0.001, 50, 32))
49 results.append(run_exp("Exp-2 (lr=0.01)", (32, 16), 0.01, 50, 32))
50 results.append(run_exp("Exp-3 (epochs=100)", (32, 16), 0.001, 100, 32))
51 results.append(run_exp("Exp-4 (batch=64)", (32, 16), 0.001, 100, 64))
52 results.append(run_exp("Exp-5 (lr=0.1)", (32, 16), 0.1, 50, 32))
53
54 pd.DataFrame(results).to_csv("keras_hyper_results.csv", index=False)
55 best_idx = pd.DataFrame(results)["f1"].idxmax()
56 print(f"\nBest configuration by F1-score: {results[best_idx]}")
57 print("Saved keras_hyper_results.csv")

```

(Part 2 of 2)

04c_ann_overfitting_regularisation.py – Overfitting & Regularisation (Keras)

```

1  """Stage C: Overfitting Analysis (Section 13) + Regularisation comparison, using Keras."""
2  import os
3  os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
4  import pandas as pd, numpy as np
5  import matplotlib
6  matplotlib.use("Agg")
7  import matplotlib.pyplot as plt
8  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
9  import tensorflow as tf
10 from tensorflow import keras
11 from tensorflow.keras import layers, regularizers
12 import joblib
13
14 tf.random.set_seed(42); np.random.seed(42)
15
16 X_train = pd.read_csv("X_train.csv"); X_test = pd.read_csv("X_test.csv")
17 y_train = pd.read_csv("y_train.csv").values.ravel()
18 y_test = pd.read_csv("y_test.csv").values.ravel()
19
20 scaler = joblib.load("final_scaler.joblib")
21 X_train_s = scaler.transform(X_train)
22 X_test_s = scaler.transform(X_test)
23 n_features = X_train_s.shape[1]
24
25 def build_model(hidden_layers, lr, l2_alpha=0.0):
26     model = keras.Sequential([layers.Input(shape=(n_features,))])
27     for units in hidden_layers:
28         reg = regularizers.L2(l2_alpha) if l2_alpha > 0 else None
29         model.add(layers.Dense(units, activation="relu", kernel_regularizer=reg))
30     model.add(layers.Dense(1, activation="sigmoid"))
31     model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
32                 loss="binary_crossentropy", metrics=["accuracy"])
33     return model
34
35 # --- Overfitting-demo model: no regularisation, 150 epochs, track full history ---
36 overfit_model = build_model((64, 32), 0.001, l2_alpha=0.0)
37 hist = overfit_model.fit(X_train_s, y_train, epochs=150, batch_size=32,
38                        validation_split=0.15, verbose=0)
39
40 train_losses = hist.history["loss"]
41 val_losses = hist.history["val_loss"]
42 train_accs = hist.history["accuracy"]

```

(Part 1 of 3)

```

43 val_accs = hist.history["val_accuracy"]
44
45 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
46 axes[0, 0].plot(train_losses, color="tab:blue"); axes[0, 0].set_title("Training Loss vs Epoch"); axes[0, 0].set_xlabel("Epoch"); axes[0, 0].set_ylabel("Loss")
47 axes[0, 1].plot(val_losses, color="tab:orange"); axes[0, 1].set_title("Validation Loss vs Epoch"); axes[0, 1].set_xlabel("Epoch"); axes[0, 1].set_ylabel("Loss")
48 axes[1, 0].plot(train_accs, color="tab:green"); axes[1, 0].set_title("Training Accuracy vs Epoch"); axes[1, 0].set_xlabel("Epoch"); axes[1, 0].set_ylabel("Accuracy")
49 axes[1, 1].plot(val_accs, color="tab:red"); axes[1, 1].set_title("Validation Accuracy vs Epoch"); axes[1, 1].set_xlabel("Epoch"); axes[1, 1].set_ylabel("Accuracy")
50 plt.tight_layout()
51 plt.savefig("plot8b_four_panel_overfitting.png", dpi=140)
52 plt.close()
53
54 min_val_epoch = int(np.argmin(val_losses))
55 print(f"Validation loss lowest at epoch {min_val_epoch+1}; rises afterward -> overfitting onset.")
56 print(f"Final training accuracy: {train_accs[-1]:.4f} | Final validation accuracy: {val_accs[-1]:.4f} | Gap: {train_accs[-1]-val_accs[-1]:.4f}")
57
58 test_preds = (overfit_model.predict(X_test_s, verbose=0) > 0.5).astype(int).ravel()
59 train_preds = (overfit_model.predict(X_train_s, verbose=0) > 0.5).astype(int).ravel()
60 final_train_acc = accuracy_score(y_train, train_preds)
61 final_test_acc = accuracy_score(y_test, test_preds)
62 print(f"Train accuracy: {final_train_acc:.4f} | Test accuracy: {final_test_acc:.4f}")
63
64 overfit_summary = pd.DataFrame({
65     "epoch": range(1, len(train_losses)+1), "train_loss": train_losses, "val_loss": val_losses,
66     "train_acc": train_accs, "val_acc": val_accs
67 })
68 overfit_summary.to_csv("overfit_epoch_log.csv", index=False)
69
70 # --- Regularisation: before vs after ---
71 def eval_model(model, epochs, batch, l2_alpha, early_stop):
72     callbacks = []
73     if early_stop:
74         callbacks.append(keras.callbacks.EarlyStopping(monitor="val_loss", patience=15, restore_best_weights=True))
75     h = model.fit(X_train_s, y_train, epochs=epochs, batch_size=batch,
76                 validation_split=0.15, callbacks=callbacks, verbose=0)
77     preds = (model.predict(X_test_s, verbose=0) > 0.5).astype(int).ravel()
78     return {
79         "accuracy": round(accuracy_score(y_test, preds), 4),
80         "precision": round(precision_score(y_test, preds), 4),
81         "recall": round(recall_score(y_test, preds), 4),
82         "f1": round(f1_score(y_test, preds), 4),
83         "epochs_actually_run": len(h.history["loss"]),
84     }

```

(Part 2 of 3)

```

85 before_model = build_model((64, 32), 0.001, l2_alpha=0.0)
86 before_reg = eval_model(before_model, 150, 32, 0.0, early_stop=False)
87 print("Before regularisation:", before_reg)
88
89 after_model = build_model((64, 32), 0.001, l2_alpha=0.01)
90 after_reg = eval_model(after_model, 150, 32, 0.01, early_stop=True)
91 print("After regularisation:", after_reg)
92
93 reg_compare = pd.DataFrame([before_reg, after_reg], index=["Before Regularisation", "After Regularisation"])
94 reg_compare.to_csv("regularisation_comparison.csv")
95 print("\nRegularisation comparison:\n", reg_compare[["accuracy", "precision", "recall", "f1"]])
96
97 # Save the final ANN (used going forward for the Fuzzy Logic stage)
98 overfit_model.save("final_ann.keras")
99 print("\nSaved final_ann.keras")

```

(Part 3 of 3)

05_fuzzy_logic.py – Fuzzy Logic Retention Decision System

```

1  """
2  Section 15 - Fuzzy Logic-Based Retention Decision Support System
3  Implemented from first principles (triangular/trapezoidal membership functions,
4  Mamdani-style rule evaluation, centroid defuzzification) so every step is
5  transparent and explainable in the report - no external fuzzy library needed.
6
7  Inputs:
8      1. Churn Risk          - from the trained ANN's predicted probability (0-1)
9      2. Customer Value      - proxy = MonthlyCharges (0-120 range in this dataset)
10     3. Tenure (months)     - 0-72 range in this dataset
11
12  Output:
13     Retention Priority score (0-100) -> mapped to Low / Medium / High
14  """
15  import numpy as np
16  import pandas as pd
17  import joblib
18  import os
19  os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
20  import matplotlib
21  matplotlib.use("Agg")
22  import matplotlib.pyplot as plt
23
24  # ----- 1. Membership functions -----
25
26  def trimf(x, a, b, c):
27      """Triangular membership function."""
28      if x <= a or x >= c:
29          return 0.0
30      if x == b:
31          return 1.0
32      if x < b:
33          return (x - a) / (b - a)
34      return (c - x) / (c - b)
35
36  def trapmf(x, a, b, c, d):
37      """Trapezoidal membership function."""
38      if x <= a or x >= d:
39          return 0.0
40      if b <= x <= c:
41          return 1.0
42      if x < b:

```

(Part 1 of 5)

```

43         return (x - a) / (b - a)
44     return (d - x) / (d - c)
45
46 # Churn risk (0-1 probability): Low / Medium / High
47 def churn_low(x):     return trapmf(x, 0.0, 0.0, 0.20, 0.40)
48 def churn_medium(x):  return trimf(x, 0.30, 0.50, 0.70)
49 def churn_high(x):    return trapmf(x, 0.60, 0.80, 1.0, 1.0)
50
51 # Customer value (monthly charges, ~18-120 in this dataset): Low / Medium / High
52 def value_low(x):     return trapmf(x, 0, 0, 35, 60)
53 def value_medium(x):  return trimf(x, 45, 70, 95)
54 def value_high(x):    return trapmf(x, 80, 100, 120, 120)
55
56 # Tenure (months, 0-72): New / Medium / Long-term
57 def tenure_new(x):    return trapmf(x, 0, 0, 6, 18)
58 def tenure_medium(x): return trimf(x, 12, 30, 48)
59 def tenure_long(x):   return trapmf(x, 36, 55, 72, 72)
60
61 # ----- 2. Output membership functions (for centroid defuzzification) -----
62 # Retention priority score scale 0-100
63 priority_domain = np.linspace(0, 100, 101)
64
65 def out_low(x):       return trapmf(x, 0, 0, 20, 45)
66 def out_medium(x):    return trimf(x, 30, 50, 70)
67 def out_high(x):      return trapmf(x, 55, 80, 100, 100)
68
69 # ----- 3. Rule base (9 rules - churn x value, tenure used as a modifier) -----
70 # Each rule returns a firing strength per output category (Low/Medium/High)
71
72 def apply_rules(risk, value, tenure):
73     cl, cm, ch = churn_low(risk), churn_medium(risk), churn_high(risk)
74     vl, vm, vh = value_low(value), value_medium(value), value_high(value)
75     tn, tm, tl = tenure_new(tenure), tenure_medium(tenure), tenure_long(tenure)
76
77     rules_fired = {"Low": [], "Medium": [], "High": []}
78
79     # Rule 1: IF risk LOW AND value LOW THEN priority LOW
80     rules_fired["Low"].append(min(cl, vl))
81     # Rule 2: IF risk HIGH AND value HIGH THEN priority HIGH
82     rules_fired["High"].append(min(ch, vh))
83     # Rule 3: IF risk HIGH AND tenure LONG THEN priority HIGH
84     rules_fired["High"].append(min(ch, tl))

```

(Part 2 of 5)

```

85 # Rule 4: IF risk MEDIUM AND value HIGH THEN priority HIGH
86 rules_fired["High"].append(min(cm, vh))
87 # Rule 5: IF risk HIGH AND value LOW THEN priority MEDIUM
88 rules_fired["Medium"].append(min(ch, vl))
89 # Rule 6: IF risk LOW AND value HIGH THEN priority MEDIUM
90 rules_fired["Medium"].append(min(cl, vh))
91 # Rule 7: IF risk MEDIUM AND value LOW THEN priority MEDIUM
92 rules_fired["Medium"].append(min(cm, vl))
93 # Rule 8: IF risk MEDIUM AND value MEDIUM THEN priority MEDIUM
94 rules_fired["Medium"].append(min(cm, vm))
95 # Rule 9: IF risk LOW AND tenure NEW THEN priority LOW (new, low-risk customer needs no urgency)
96 rules_fired["Low"].append(min(cl, tn))
97
98 strength = {k: max(v) if v else 0.0 for k, v in rules_fired.items()}
99 return strength
100
101 def defuzzify(strength):
102     """Centroid (center-of-gravity) defuzzification over the output domain."""
103     agg = np.zeros_like(priority_domain, dtype=float)
104     for i, x in enumerate(priority_domain):
105         agg[i] = max(
106             min(strength["Low"], out_low(x)),
107             min(strength["Medium"], out_medium(x)),
108             min(strength["High"], out_high(x)),
109         )
110     if agg.sum() == 0:
111         return 0.0
112     return float(np.sum(priority_domain * agg) / np.sum(agg))
113
114 def retention_priority(risk, value, tenure):
115     strength = apply_rules(risk, value, tenure)
116     score = defuzzify(strength)
117     if score < 35:
118         label = "Low"
119     elif score < 65:
120         label = "Medium"
121     else:
122         label = "High"
123     return score, label
124
125 # ----- 4. Apply to the test set using ANN churn-risk predictions -----
126 import tensorflow as tf

```

```

127 from tensorflow import keras
128 ann = keras.models.load_model("final_ann.keras")
129 scaler = joblib.load("final_scaler.joblib")
130
131 X_test = pd.read_csv("X_test.csv")
132 y_test = pd.read_csv("y_test.csv").values.ravel()
133 X_test_s = scaler.transform(X_test)
134
135 churn_prob = ann.predict(X_test_s, verbose=0).ravel()
136
137 results = pd.DataFrame({
138     "ActualChurn": y_test,
139     "ChurnRiskProb": churn_prob,
140     "MonthlyCharges": X_test["MonthlyCharges"].values,
141     "Tenure": X_test["tenure"].values,
142 })
143
144 scores, labels = [], []
145 for _, row in results.iterrows():
146     s, l = retention_priority(row["ChurnRiskProb"], row["MonthlyCharges"], row["Tenure"])
147     scores.append(s); labels.append(l)
148
149 results["RetentionScore"] = scores
150 results["RetentionPriority"] = labels
151
152 results.to_csv("fuzzy_retention_results.csv", index=False)
153
154 print("Retention priority distribution on test set:")
155 print(results["RetentionPriority"].value_counts())
156
157 print("\nMean churn probability by assigned priority:")
158 print(results.groupby("RetentionPriority")["ChurnRiskProb"].mean())
159
160 print("\nSample decisions:")
161 print(results.sample(8, random_state=1).round(3).to_string(index=False))
162
163 # Visualise: retention priority vs actual churn outcome (validation of fuzzy logic)
164 plt.figure(figsize=(6, 4))
165 order = ["Low", "Medium", "High"]
166 actual_churn_rate = results.groupby("RetentionPriority")["ActualChurn"].mean().reindex(order) * 100
167 actual_churn_rate.plot(kind="bar", color=["#2ecc71", "#f39c12", "#e74c3c"])
168 plt.title("Actual Churn Rate by Fuzzy Retention Priority (Test Set)")

```

(Part 4 of 5)

```

169 plt.ylabel("Actual Churn Rate (%)"); plt.xticks(rotation=0)
170 plt.tight_layout()
171 plt.savefig("plot9_fuzzy_validation.png", dpi=140)
172 plt.close()
173
174 print("\nActual churn rate by assigned priority (validates the fuzzy system):")
175 print(actual_churn_rate.round(1))

```

(Part 5 of 5)

Appendix B — Actual Console Output From Running the Code

Captured directly from executing each script in sequence — these are the real printed results the report's numbers are drawn from (minor library convergence warnings omitted for readability).

Output: 01_data_prep.py

```
=== DIMENSIONS ===
(7043, 21)

=== DTYPES (before fix) ===
customerID      str
gender          str
SeniorCitizen   int64
Partner         str
Dependents      str
tenure          int64
PhoneService    str
MultipleLines   str
InternetService str
OnlineSecurity  str
OnlineBackup    str
DeviceProtection str
TechSupport     str
StreamingTV     str
StreamingMovies str
Contract        str
PaperlessBilling str
PaymentMethod   str
MonthlyCharges  float64
TotalCharges    str
Churn           str
dtype: object

=== MISSING VALUES ===
TotalCharges    11
dtype: int64

Rows with missing TotalCharges - tenure values:
[0]

Duplicate rows: 0
Duplicate customerIDs: 0
tenure: 0 potential outliers (IQR method)
MonthlyCharges: 0 potential outliers (IQR method)
TotalCharges: 0 potential outliers (IQR method)

Categorical columns to encode: ['gender', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
'PaymentMethod']

Final modelling dataframe shape: (7043, 33)

Saved telco_clean.csv (for EDA) and telco_model_ready.csv (for modelling)
```

Output: 02_eda.py

Overall churn rate: 26.54%

/home/claude/churn_project/02_eda.py:19: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x="ChurnLabel", palette=["#2ecc71", "#e74c3c"])
```

Churn rate by contract:

Contract	
Month-to-month	42.7
One year	11.3
Two year	2.8

Name: Yes, dtype: float64

Churn rate by tenure group:

TenureGroup	
0-1yr	47.4
1-3yr	25.5
3yr+	11.9

Name: Yes, dtype: float64

Churn rate by charge band:

ChargeBand	
Low	15.9
Medium	29.7
High	34.1

Name: Yes, dtype: float64

Churn rate by payment method:

PaymentMethod	
Bank transfer (automatic)	16.7
Credit card (automatic)	15.2
Electronic check	45.3
Mailed check	19.1

Name: Yes, dtype: float64

All EDA plots saved.

Output: 03_ml_models.py

=== Logistic Regression ===

accuracy: 0.8070
precision: 0.6584
recall: 0.5668
f1: 0.6092
roc_auc: 0.8418
Confusion matrix [[TN,FP],[FN,TP]]: [[925, 110], [162, 212]]

=== Decision Tree ===

accuracy: 0.8006
precision: 0.6667
recall: 0.4973
f1: 0.5697

```
roc_auc: 0.8219
Confusion matrix [[TN,FP],[FN,TP]]: [[942, 93], [188, 186]]

=== Random Forest ===
accuracy: 0.8041
precision: 0.6667
recall: 0.5241
f1: 0.5868
roc_auc: 0.8425
Confusion matrix [[TN,FP],[FN,TP]]: [[937, 98], [178, 196]]
```

Top 10 predictors (Random Forest importance):

```
tenure                0.183643
TotalCharges          0.160776
MonthlyCharges        0.121749
InternetService_Fiber optic 0.065881
Contract_Two year     0.060685
PaymentMethod_Electronic check 0.059232
OnlineSecurity_Yes    0.034077
Contract_One year     0.033241
TechSupport_Yes       0.028200
PaperlessBilling      0.022867
dtype: float64
```

Saved ml_results.json, top_predictors.csv, and train/test splits.

Output: 04a_ann_architecture.py

```
ANN-1 (16) {'architecture': 'ANN-1 (16)', 'hidden_layers': '(16,)', 'learning_rate': 0.001, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7892, 'precision': 0.6156, 'recall': 0.5481, 'f1': 0.5799, 'train_accuracy': 0.8188, 'gap': 0.0296}
ANN-2 (32,16) {'architecture': 'ANN-2 (32,16)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.001, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7644, 'precision': 0.5633, 'recall': 0.5, 'f1': 0.5297, 'train_accuracy': 0.8435, 'gap': 0.0791}
ANN-3 (64,32) {'architecture': 'ANN-3 (64,32)', 'hidden_layers': '(64, 32)', 'learning_rate': 0.001, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7665, 'precision': 0.5684, 'recall': 0.5, 'f1': 0.532, 'train_accuracy': 0.8676, 'gap': 0.1011}
```

Saved keras_arch_results.csv

Output: 04b_ann_hyperparameters.py

```
Exp-1 (base) {'experiment': 'Exp-1 (base)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.001, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7786, 'precision': 0.5934, 'recall': 0.5267, 'f1': 0.5581}
Exp-2 (lr=0.01) {'experiment': 'Exp-2 (lr=0.01)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.01, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7665, 'precision': 0.5728, 'recall': 0.4733, 'f1': 0.5183}
Exp-3 (epochs=100) {'experiment': 'Exp-3 (epochs=100)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.001, 'epochs': 100, 'batch_size': 32, 'accuracy': 0.7622, 'precision': 0.5543, 'recall': 0.5321, 'f1': 0.543}
Exp-4 (batch=64) {'experiment': 'Exp-4 (batch=64)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.001, 'epochs': 100, 'batch_size': 64, 'accuracy': 0.7722, 'precision': 0.5796, 'recall': 0.516, 'f1': 0.546}
Exp-5 (lr=0.1) {'experiment': 'Exp-5 (lr=0.1)', 'hidden_layers': '(32, 16)', 'learning_rate': 0.1, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7346, 'precision': 0.0, 'recall': 0.0, 'f1': 0.0}
```

```
Best configuration by F1-score: {'experiment': 'Exp-1 (base)', 'hidden_layers': '(32, 16)',  
'learning_rate': 0.001, 'epochs': 50, 'batch_size': 32, 'accuracy': 0.7786, 'precision': 0.5934,  
'recall': 0.5267, 'f1': 0.5581}  
Saved keras_hyper_results.csv
```

Output: 04c_ann_overfitting_regularisation.py

```
Validation loss lowest at epoch 2; rises afterward -> overfitting onset.  
Final training accuracy: 0.9330 | Final validation accuracy: 0.7435 | Gap: 0.1895  
Train accuracy: 0.8981 | Test accuracy: 0.7622  
Before regularisation: {'accuracy': 0.7559, 'precision': 0.5441, 'recall': 0.4947, 'f1': 0.5182,  
'epochs_actually_run': 150}  
After regularisation: {'accuracy': 0.7949, 'precision': 0.6324, 'recall': 0.5428, 'f1': 0.5842,  
'epochs_actually_run': 50}
```

Regularisation comparison:

	accuracy	precision	recall	f1
Before Regularisation	0.7559	0.5441	0.4947	0.5182
After Regularisation	0.7949	0.6324	0.5428	0.5842

Saved final_ann.keras

Output: 05_fuzzy_logic.py

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR  
I0000 00:00:1789238436.073101 32169 port.cc:153] oneDNN custom operations are on. You may see slightly  
different numerical results due to floating-point round-off errors from different computation orders. To  
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.  
I0000 00:00:1789238436.073418 32169 cudart_stub.cc:31] Could not find cuda drivers on your machine,  
GPU will not be used.  
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR  
I0000 00:00:1789238437.485027 32169 port.cc:153] oneDNN custom operations are on. You may see slightly  
different numerical results due to floating-point round-off errors from different computation orders. To  
turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.  
I0000 00:00:1789238437.485377 32169 cudart_stub.cc:31] Could not find cuda drivers on your machine,  
GPU will not be used.  
Retention priority distribution on test set:  
RetentionPriority  
Low 794  
Medium 463  
High 152  
Name: count, dtype: int64  
  
Mean churn probability by assigned priority:  
RetentionPriority  
High 0.813401  
Low 0.101395  
Medium 0.199514  
Name: ChurnRiskProb, dtype: float32  
  
Sample decisions:
```

ActualChurn	ChurnRiskProb	MonthlyCharges	Tenure	RetentionScore	RetentionPriority
1	0.760	78.05	27	0.000	Low
0	0.000	59.00	28	22.500	Low
1	0.772	70.00	2	0.000	Low

0	0.000	19.95	51	17.312	Low
1	0.000	19.30	2	17.312	Low
0	0.002	23.75	72	17.312	Low
0	0.026	71.45	72	0.000	Low
0	0.000	89.80	54	50.000	Medium

Actual churn rate by assigned priority (validates the fuzzy system):

RetentionPriority

Low 20.3

Medium 27.9

High 55.3

Name: ActualChurn, dtype: float64